UNIVERSITAT POLITÈCNICA DE VALÈNCIA

MIARFID Official Master's Degree in Artificial Intelligence, Pattern Recognition and Digital Imaging

Master Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Universidad Politécnica de Valencia

Agentes para Catan con heurísticas y modelos de lenguaje

Herramientas y Aplicaciones de la Inteligencia Artificial

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital

Gómez Corral, Miquel Pachecho Millan, Andrés

CAPÍTULO 1
Introducción

En este trabajo contamos cómo hemos creado y evaluado un agente para *PyCatan*. Hemos seguido dos caminos: reglas heurísticas (más rápidas y predecibles) y decisiones apoyadas por modelos de lenguaje (LLM). La idea principal es comprobar qué funciona mejor en cada tipo de decisión del juego y hasta qué punto compensa usar LLM en una simulación con muchas partidas.

1.1 Alcance y estado actual

Se han completado los experimentos de la parte heurística y se ha dejado preparada toda la parte LLM para tres proveedores (Ollama, AWS Bedrock y API UPV). A día de hoy tenemos:

- Resultados completos del agente heurístico frente a random y estándar.
- Resultados **preliminares** de un *smoke test* del agente LLM con Ollama.
- Integración lista para Bedrock y UPV, pero faltan pruebas completas porque aún no tenemos credenciales.

CAPÍTULO 2

Parte 1: Agente heurístico

2.1 Diseño del agente

El agente heurístico se basa en reglas simples y rápidas de ejecutar. La ventaja es que su comportamiento es fácil de entender y depurar.

2.1.1. Colocación inicial

En `on_game_start`, el agente revisa los nodos válidos y da prioridad a:

- Buena probabilidad total de producción en terrenos cercanos.
- Variedad de recursos para no quedarse atascado.
- Casillas potentes (6 y 8).

2.1.2. Priorización de recursos y construcción

En la fase de construcción sigue este orden:

1. Construir ciudad si es posible.
2. Construir poblado en el nodo con mejor potencial de producción.
3. Construir carretera para mantener expansión.
4. Comprar carta de desarrollo si no hay mejor opción estructural.

También se corrigió el descarte por ladrón para evitar casos donde la partida podía quedarse bloqueada.

2.2 Resultados del benchmark heurístico

Con base en `results_heuristic.json`:

oprule extbfRival	**Partidas**	**Victorias**	**Tasa de victoria**	**Puntos medios**	**Turnos medios**
Random	100	44	44.00 %	7.07	444.65
Estándar	100	19	19.00 %	5.00	281.80

Tabla 2.1: Rendimiento del agente heurístico frente a random y agentes estándar.

Lectura rápida El agente mejora claramente frente a random y compite de forma razonable frente a los agentes estándar. Aun así, hay margen de mejora, sobre todo en decisiones donde conviene adaptarse mejor al contexto de la partida.

CAPÍTULO 3

Parte 2: Integración de modelos de lenguaje

3.1 Cómo se conecta el agente con los LLM

Se implementó una única interfaz para tres proveedores:

- Ollama local.
- AWS Bedrock.
- API UPV.

La entrada y salida se definieron en JSON para evitar respuestas ambiguas. El agente LLM extiende al heurístico, así que si hay timeout o respuesta inválida, vuelve automáticamente al comportamiento heurístico.

3.1.1. Decisiones que toma el LLM

Se delegaron dos decisiones, como pide la práctica:

1. `on_game_start`: selección de nodo y carretera inicial.
2. `on_build_phase`: selección de acción de construcción (city/town/road/card/none).

3.1.2. Estado resumido para ahorrar tokens

En vez de enviar toda la partida, enviamos solo lo importante:

- Nodos válidos y resumen de terrenos cercanos (tipo y probabilidad) para la apertura.
- Recursos en mano y acciones legales para construir.
- Restricciones explícitas sobre formato de salida.

Con esto se reducen tokens, tarda menos y baja el número de respuestas mal formadas.

3.2 Prompts diseñados (3 variantes)

3.2.1. Prompt A: formato estricto

```
Eres un asistente de Catan. Devuelve solo JSON valido.
Apertura: {"node_id": int, "road_to": int}
Construccion: {"building": "city|town|road|card|none", ...}
Elige solo acciones legales dentro de las opciones dadas.
```

3.2.2. Prompt B: formato + validación interna

```
Piensa si la accion es legal y devuelve solo JSON valido.
No escribas texto fuera del JSON.
Si no hay accion legal, devuelve {"building": "none"}.
```

3.2.3. Prompt C: formato + objetivo táctico

```
Objetivo: mejorar produccion a medio plazo y capacidad de expansion.
Prioriza acciones legales que aumenten ingresos de recursos.
Devuelve solo JSON, sin explicaciones.
```

3.3 Resultados LLM y comparación de modelos

extbfEstado de resultados: de momento solo hay resultados completos de un *smoke test* con Ollama. Bedrock y UPV están listos a nivel técnico, pero falta ejecutar pruebas completas cuando tengamos credenciales.

3.3.1. Resultado preliminar disponible (Ollama)

Con base en `results_llm_smoke.json`:

oprule extbfRival	**Partidas**	**Victorias**	**Tasa de victoria**	**Puntos medios**	**Turnos medios**
Random	4	1	25.00 %	7.50	523.00
Estándar	4	3	75.00 %	7.00	337.50

Tabla 3.1: Resultado preliminar del agente LLM con Ollama (muestra pequeña).

3.3.2. Comparativa de modelos/proveedores (pendiente de completar)

oprule extbfProveedor/Modelo	**Latencia media**	**Errores de formato**	**Calidad de decision**	**Esta**
Ollama / llama3.2:3b	~90 s por partida (smoke)	0 en smoke	Media (muestra pequena)	Parc
Bedrock / [PENDIENTE]	[MOCK] 0.0s	[MOCK] 0	[MOCK] N/A	Falta eje
UPV API / [PENDIENTE]	[MOCK] 0.0s	[MOCK] 0	[MOCK] N/A	Falta eje

Tabla 3.2: Comparativa provisional de modelos. Filas con [MOCK]/[PENDIENTE] deben reemplazarse tras experimentación real.

Nota importante Los valores [MOCK] no son resultados reales. Están solo para dejar la tabla preparada y rellenarla cuando se hagan las pruebas pendientes.

CAPÍTULO 4

Parte 2.4: Análisis y conclusiones

4.1 Discusión

La evidencia disponible confirma que:

- El enfoque heurístico es estable, rápido y permite lanzar muchas partidas sin problemas.
- El enfoque LLM es más flexible, pero cada llamada tarda más.
- Si se simulan muchas partidas, ese tiempo extra se nota mucho.

4.2 Viabilidad de LLM en este contexto

El uso de LLM tiene más sentido en decisiones importantes que se toman pocas veces (por ejemplo, la apertura). Para decisiones repetitivas, sigue siendo mejor usar heurísticas o un enfoque híbrido.

4.3 Qué falta para cerrar la parte experimental

1. Ejecutar benchmarks comparables con Bedrock y UPV en las mismas condiciones.
2. Medir latencia por llamada y por partida para cada proveedor/modelo.
3. Medir errores de formato JSON y tasa de fallback al agente heurístico.
4. Comparar explícitamente Prompt A/B/C con igualdad de presupuesto de partidas.

CAPÍTULO 5

Parte 3.1: Uso de herramientas de IA en el desarrollo

Durante el desarrollo usamos herramientas de IA generativa para apoyar tareas de programación y redacción. En concreto, nos ayudaron en:

- Automatizar scripts de benchmark con progreso y exportación de resultados.
- Diseñar una capa de abstracción de proveedores LLM (Ollama/Bedrock/UPV).
- Redactar borradores de secciones y tablas en LaTeX.

En general, estas herramientas aceleraron el trabajo y evitaron errores mecánicos. Aun así, las decisiones finales de diseño, la interpretación de resultados y la versión final del texto las hizo el equipo.

5.1 Aportación humana pendiente de redactar

extbfSección para completar manualmente por los autores:

- Reparto exacto de tareas entre miembros del equipo.
- Valoración personal crítica sobre limitaciones encontradas con cada herramienta de IA.
- Decisiones de diseño que se descartaron y justificación final.